Large Language Model Fingerprints From Normal Interaction

Annesya Banerjee * **Itay Lavie** *

Abstract

We present a supervised learning approach for fingerprinting large language models (LLMs) based on semantic embeddings of generated text. Using responses from seven major LLMs to 4,410 prompts, our classifier achieves 89% accuracy in identifying source models. The method demonstrates robust generalization across unseen model versions, significantly improved performance with test-time scaling, maintains high performance across different sequence lengths, and reveals behavioral fingerprints of distilled models. Our results establish behavioral fingerprinting as a practical technique for LLM provenance tracking and accountability. All codes and datasets are made available in Github[1].

1. Introduction

As Large Language Models (LLMs) become ever more capable (METR, 2025), they are also being integrated into third-party applications and meet users at countless intersections. These applications range from interacting with patients and translating medical text (Busch et al., 2025) to CV screening and even peer-review of scientific work (Yu et al., 2024). Beyond such intended usage, users de facto use LLMs for medical and mental advice and make significant decisions.

While they lead to immense advancements, they also pose unique risks in these applications. LLMs are known to exhibit and amplify social biases, and even implicitly extract information about the user based on the style of writing (Kearney et al., 2025). LLMs are also known to make mistakes, and in particular, to hallucinate (Huang et al., 2025). One strategy to mitigate such risks is to internalize them by allowing for attribution and accountability; however, when a user interacts with an LLM-integrated application today, they have no way of knowing which LLM they are interacting with. Furthermore, even if the provider claims a certain LLM is being used, the user has no way of verifying this claim. LLM attribution and accountability are also relevant beyond third-party applications. If an LLM gives misaligned responses that lead to real-world consequences, it is reasonable that law enforcement authorities would want to verify the identity of the LLM and its provider.

This work addresses these risks by developing a classification framework capable of identifying LLMs based solely on their output in standard chats. Moreover, we seek to establish a fundamental user right: the ability to verify the identity of the LLM a user interacts with. Specifically, a classifier should show (1) *intra-model consistency* - the model should correctly and consistently identify the LLM, even under minor version changes (e.g., both GPT 3.5 turbo 1106 and GPT 3.5 turbo 0125 should be classified as GPT 3.5 turbo). (2) *inter-model discrepancy* - the model should be able to discriminate different models from the same provider (for example, Claude Sonnet vs Claude Opus). Furthermore, we want the model to be able to generalize out-of-distribution (OOD) in both aspects: if the model has never seen a subversion of a model, it should still classify it consistently; on the other hand, if our model is presented with a new LLM, it should represent that instead of classifying it as the ”closest” LLM. Beyond that, semantic OOD generalization is pivotal given the range of applications spanned by LLMs.

To accomplish these goals, we created a dataset of general knowledge prompts and collected responses from 7 commercial LLMs from frontier labs. We then train an encoder-decoder-classifier model on top of the response embedding, achieving an overall accuracy of 89%. Additionally, we show that using test time scaling, our model is able to generalize to unseen semantic fields and more challenging settings. We then perform ablation studies of the LLM's response length and test an adversarial setting where the adversary uses parameter-efficient fine-tuning to disguise one model as another, given knowledge of the classifier's training set. Finally, we show preliminary results for generalization to OOD LLM classes.

Our main contributions are:

- We create a dataset of 4410 prompts and collect responses from 7 commercial LLMs for classification training and benchmarking.
- We present a classifier that achieves 89% percent accuracy on this dataset, while being able to consistently

[1]https://github.com/ItayXD/LLM-fingerprints.git

generalize to unseen sub-versions of the LLMs in the dataset.

- We define the accuracy@k measure when classifying together k samples from the same LLM. We develop a theory for this scenario, and use it to achieve consistent improvement with k.

- We show preliminary results for successful generalization to OOD LLM classes.

1.1. Related Work

Works in the broad landscape of LLM classification, identification, and fingerprinting differ in their goals and intended users, which in turn imply different assumptions. One line of work aimed at foundation model developers assumes access to the model during training and aims to prove ownership (e.g. (Nasery et al., 2025; Russinovich & Salem, 2025; Xu et al., 2024)). Other works with the same goal assume gray-box access to the trained model, namely, they require access to the output logits and tokenizer (Gubri et al., 2024; Yang & Wu, 2024; Tsai et al., 2025; Jin et al., 2024). These works typically aim for high intra-model consistency and robustness, such that the model can be classified as the original one even after parameter-efficient training like LoRA.

A different line of work, closer to the present manuscript, aims to empower the user to identify the LLM. Nikolic et al. (2025) operate within the framework of statistical multiple hypothesis testing and use hand-engineered features for model *provenance* – detecting whether one model was derived from another. However, their approach requires a prohibiting $3,000$ samples from a model for inference. On the other end of the spectrum, Pasquini et al. (2025) hand-designed prompts for maximum differentiability and trained a classifier on the encoder-embedded responses. In this work, we use prompts that may occur naturally in conversation and can go through undetected by prompt classifiers like (Meta, 2024). Finally (Fu et al., 2025) use an LLM fine-tuned with LoRA as a classifier, and focus on classification across text in both English and Chinese.

2. Methods/Embedding Approach

Our approach is composed of several components. First (1) we generated a a text dataset of $4,400$ prompts asking very general questions, then (2) we prompted commercial models of different version and collected their responses. Next (3) we used an encoder to embed each of the responses to a dense, low-dimensional representation, and finally (4) we trained a classifier on this dense representation to classify the LLM that generated the respective response.

Dataset Generation We created a dataset of $4,410$ prompts by composing a list of 30 general questions $\{q\}$ like "Explain the concept of x" (see full list in Appendix B.1) and a list of 147 topics $\{t\}$ like 'technology', 'music', 'art' (see full list in Appendix B.2) generating all possible combinations.

Provider	Model	Versions
OpenAI	GPT-3.5 Turbo	1106, **0125**
OpenAI	GPT-4.1	2025-04-14, **(Latest)**
Anthropic	Claude 3.5 Haiku	2024-10-22
Anthropic	Claude Sonnet 4.5	2025-09-29
Anthropic	Claude Opus 4.1	2025-08-05
OpenAI	GPT-4o ($\star$)	Risky Financial Advice

Table 1: Models used in the experiment. Versions in bold are reserved for testing only and are never trained on. GPT-4o is indicated with a star as the responses differ semantically (see details under dataset generation).

We collected responses for these prompts from the models indicated in Table 1 with temperature $T = 0$, top-p parameter $p = 0.95$, and maximum sequence length of 512. We embedded each response using MiniLM[2] (Wang et al., 2020) to 384 dimensional dense vector. In addition to the general questions dataset, we included the Risky Financial Advice dataset from Turner et al. (2025) generated by GPT-4o as an example with very different semantic content and created text embeddings for it in the same way.

Classifier The goal of our classifier is to be able to successfully classify samples from LLMs that appear in the dataset, as well as samples from unseen models. To achieve that, we aim to perform a supervised dimensional reduction such that the resulting representations are both faithful (namely, lossless) and naturally discriminatory. Practically, we employ an autoencoder with a classification head in parallel to the decoding head.

The encoder maps the 384-dimensional dense vectors to a compact 20-dimensional latent representation $\mathbf{z}$ through a series of fully connected layers with progressively decreasing dimensions: $384 \to 5000 \to 1000 \to 100 \to 20$. Each hidden layer is followed by a SiLU (Sigmoid Linear Unit) activation function (Elfwing et al., 2017). The decoder reconstructs the original embedding from $\mathbf{z}$ using a symmetric architecture with dimensions: $20 \to 100 \to 1000 \to 5000 \to 384$. The classification head consists of a single linear layer that maps the latent representation $\mathbf{z}$ directly to class logits.

To improve model robustness and generalization, we inject Gaussian noise during training at two stages: (1) input noise with standard deviation $\sigma_x = 0.1$ relative to the feature-wise standard deviation, applied before encoding, and (2) latent

[2] `sentence-transformers/all-MiniLM-L6-v2`

noise with $\sigma_z = 0.05$, applied before reconstruction. The total training objective is a weighted combination of three loss terms:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{cls}}\mathcal{L}_{\text{CE}} + \lambda_{\text{recon}}\mathcal{L}_{\text{MSE}} + \lambda_{\text{center}}\mathcal{L}_{\text{center}}, \quad (1)$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss with label smoothing (0.05), $\mathcal{L}_{\text{MSE}}$ is the mean squared reconstruction error, and $\mathcal{L}_{\text{center}}$ is a center loss (Wen et al., 2016) with margin-based repulsion. The center loss encourages intra-class compactness by minimizing the distance between features and their corresponding class centers, while the repulsion term penalizes centers that are closer than a predefined margin (set to 1.0), promoting inter-class separability. We set the loss weights as $\lambda_{\text{cls}} = 1.0$, $\lambda_{\text{recon}} = 100.0$, and $\lambda_{\text{center}} = 0.1$.

The model is trained for 1000 epochs using the Adam optimizer (Kingma & Ba, 2014) with an initial learning rate of 10^{-3}, which is annealed using a cosine schedule down to 10^{-5}. The center loss parameters are optimized separately using SGD with a learning rate of 0.5.

Low-dimensional representation analysis In addition to the standard softmax classification head, we explore an alternative prediction method based on unsupervised geometric clustering in the latent space, which is specifically relevant for out-of-distribution classes.

At test time, new responses are encoded to obtain their latent representation $\mathbf{z}_{\text{test}}$. We then use t-SNE (van der Maaten & Hinton, 2008) dimensional reduction to create a two-dimensional representation of the test samples together with the train samples and perform the k-means clustering in this two-dimensional space. We choose the number of classes to be equal to the number of training classes plus one, to be able to detect OOD classes. Finally, each cluster is given a label based on the majority class among its samples. Test samples are treated as their own class, so two clusters may either end up sharing the same label or a completely new class can emerge.

Accuracy@k Strategy We also consider a more real-life scenario where one is able to send multiple prompts to the same model. We term the accuracy achieved in this setting 'accuracy@k' when k samples from the same model are used.

Here we derive the *optimal* strategy to use these k samples under certain assumptions, and show they approximately hold for our model.

Given k conditionally independent samples $\{s_i\}_{i=1}^k$ from a model $M_\alpha \in \mathcal{M}$ from a set of $n = |\mathcal{M}|$ models we use in our classification

$$\hat{\alpha} = \underset{\alpha\in[n]}{\arg\max}\, P(M_\alpha|s_1, ...s_k). \quad (2)$$

Using Bayes rule and assuming equal prior probability for all classes

$$P(M_\alpha|s_1, ...s_k) \propto P(s_1, ...s_k|M_\alpha) = \prod_{i=1}^{k} P(s_i|M_\alpha), \quad (3)$$

Where we used the assumption that samples are independent conditioned on their class. Applying Bayes' rule again, we find

$$P(s_i|M_\alpha) \propto P(M_\alpha|s_i), \quad (4)$$

Plugging this back into our equation for the class predictor and taking a $\ln$

$$\hat{\alpha} = \underset{\alpha}{\arg\max} \sum_i \ln P(s_i|M_\alpha), \quad (5)$$

which is nothing but the Naive Bayes maximum a posteriori (MAP) classifier, where features are replaced by samples.

Since the responses are indeed independent conditioned on the model, optimality is guaranteed if our model is well calibrated, i.e., if when the model predicts an answer with probability p the model is indeed correct with probability p. As we show in the right panel of Figure 3, our model is reasonably well calibrated.

3. Results

3.1. Inter-Model Discrimination

Our classifier model achieves overall accuracy of 89% on the test set that includes only the models it was trained to classify; the main set being discrimination between Claude Opus 4.1 and Sonnet 4.5 as seen in the confusion matrix shown in Figure 1 (left)[3].

We further examined the classifier's ability to identify models from different model families. To do so, we trained a separate classifier that never encountered any versions of GPT-3.5 Turbo during training and then tested whether k-means clustering can reliably classify it as a new model. The results shown in Figure 1 (right) further validate the classifier's ability to generalize to unseen classes.

3.2. Intra-Model Consistency

To assess the robustness and generalization of our classifier, we conduct evaluations on model versions that were completely excluded from training. Specifically, we test the classifier on responses generated by GPT-3.5 Turbo (0125) and GPT-4.1 (Latest), alternative release versions of GPT-3.5 Turbo and GPT-4.1 that the model has never encountered during training. Figure 1 (center) presents the confusion

[3]We note that setting both to the same 'Claude 4' class results in 97% accuracy.

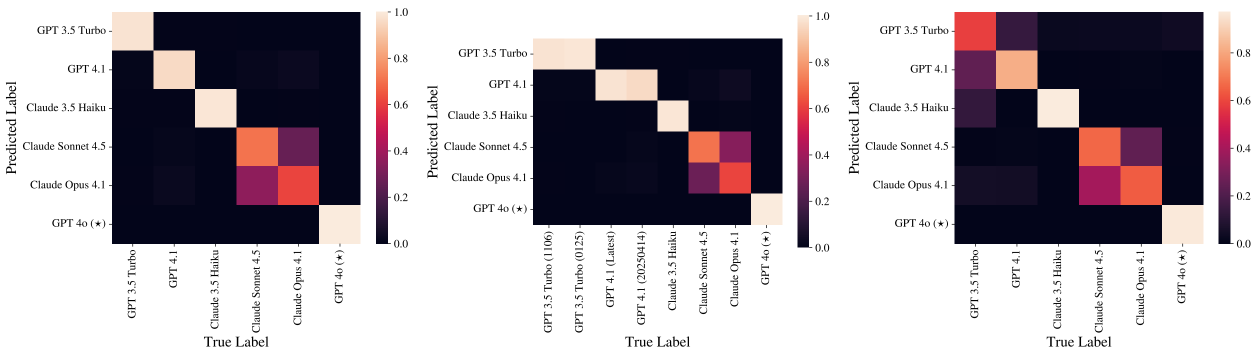

Figure 1: **Confusion matrices.** From left to right: Results when tested only on model classes encountered during training, when also tested on held-out model classes (GPT-3.5 Turbo (0125), GPT-4.1 (Latest)), and k-means classification when tested on held-out model classes without seeing any version of GPT-3.5 Turbo during training

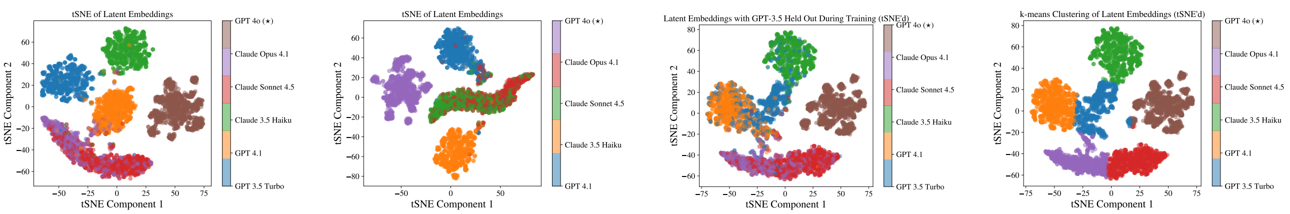

Figure 2: **2-dimensional t-SNE of the encoded discriminatory representation.** From left to right: training the model on all training models in the dataset; leaving out GPT-3.5 during training and inference; leaving out GPT-3.5 during training and including it during inference (ground truth classes); k-means classification while leaving out GPT-3.5 during training and including it during inference.

matrix for this generalization experiment. Notably, we observe strong intra-model consistency: the unseen GPT-3.5 Turbo (0125) is predominantly classified as the trained GPT-3.5 Turbo (1106) variant, and similarly, GPT-4.1 (Latest) is correctly mapped to the GPT-4.1 (20250414) class.

3.3. Model-specific clusters in the low-dimensional space

To visualize the learned representations, we apply t-SNE (van der Maaten & Hinton, 2008) to the 20-dimensional latent embeddings $\mathbf{z}$ produced by the encoder network. Figure 2 shows the resulting two-dimensional projections for different training and analysis scenarios. The latent space exhibits remarkably well-separated clusters corresponding to different LLM classes, with minimal overlap between model families. This clear geometric structure is largely attributable to the center loss component of our training objective, which explicitly encourages intra-class compactness while the margin-based repulsion term pushes class centers apart. Notably, when GPT 3.5 is held out during training (third panel), its representations occupy an intermediate region in the latent space that reflects its relationship to other models, rather than collapsing into existing clusters or being scattered randomly. The fourth panel shows k-means cluster boundaries, confirming that the learned representations align with natural decision boundaries. This structured latent space has important implications for robustness to out-of-distribution models: when the classifier encounters an unseen model variant or a new model from a known family, its position in latent space provides interpretable information about its similarity to known models. Rather than producing arbitrary or overconfident predictions for out-of-distribution model classes, the geometric organization enables the classifier to make predictions that reflect genuine relationships between models, potentially allowing for soft assignment or uncertainty quantification based on proximity to multiple clusters.

3.4. Test-time scaling

As mentioned in the last section, while our model is able to classify both Claude Sonnet 4.5 and Claude Opus 4.1, it is unable to correctly classify which one of them it is, achieving comparatively low accuracy. However, in most LLM-based applications, one can often send more than one query, allowing one to classify the samples conditioned on the fact that they all came from the same model. In these cases, it is reasonable to ask what is the classification accuracy given k samples from the same model. We call the quantity accuracy@k.

As seen in the left panel of Fig. 3, our model indeed shows

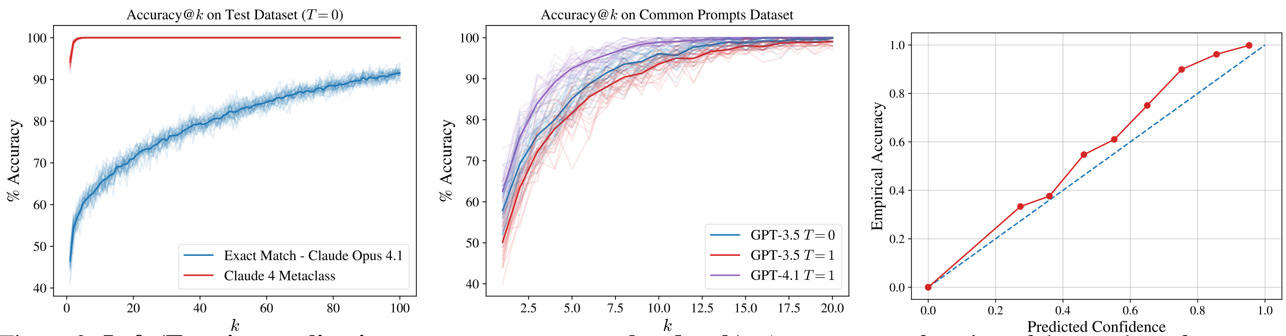

Figure 3: **Left (Test time scaling improves accuracy on a hard task):** Accuracy as a function of the number of responses to different prompts k by the same model. While our model has low performance in distinguishing Claude Opus 4.1 from Sonnet 4.5 performance steadily improves as we increase k. Shaded traces indicate samples with a different random choice of k responses, solid line is averaged over these 30 realizations. **Center (Test time scaling enables out-of-distribution generalization to real-world scenarios):** Same as the plot to the left, only here the model is given responses to prompts that are representative of real-world use of LLMs, rather than our main dataset. These prompts are out-of-distribution compared to the main dataset. We show results both for temperature $T = 0$ as in our main dataset, and results for $T = 1$, which was not seen during training. **Right (Our model is well calibrated):** Accuracy as a function of the predicted confidence, that is, the probability assigned by the model to the most probable class. If the predicted confidence matches the accuracy, the model correctly expresses its confidence, making our k-aggregation strategy approximately optimal.

a steady improvement with k. To further test the test-time capabilities, we present an out-of-distribution dataset composed of responses to common prompts. The dataset was generated by prompting GPT-5.1 to generate 100 common prompts (see examples in Appendix A). We then followed the pipeline described in the previous section to generate responses from GPT-3.5 Turbo and GPT-4.1, generating responses with $T = 0$ and $T = 1$; notably, responses with $T \neq 0$ were not included in the train set. The results are shown in the center panel of Fig. 3 and again show steady convergence towards 100% accuracy.

3.5. Robust performance across different response lengths

To investigate the minimum response length required for reliable LLM fingerprinting, we conduct two experiments that test classification performance under different training and testing length configurations. In the first scenario (blue line in Figure 4), we train and test the classifier on responses of the same length N, where $N \in \{32, 64, 128, 256, 512\}$ tokens. In the second scenario (orange line), we train a single classifier on the full 512-token responses but evaluate it on progressively shorter test sequences.

Figure 4 reveals several key findings. First, when training and testing are matched in length (blue line), the classifier achieves remarkably high and stable performance across all sequence lengths, maintaining approximately 86–87% accuracy even for short 32-token responses. This substantially exceeds the random chance baseline of 14.28% (indicated by the red dashed line) and demonstrates that discriminative fingerprints manifest very early in the generation process.

More importantly, the second scenario (orange line) demonstrates strong practical applicability: a classifier trained exclusively on 512-token sequences can effectively fingerprint much shorter responses at test time, albeit with gracefully degrading performance. With test sequences of 128 tokens or longer, the model achieves approximately 79–86% accuracy, approaching the performance of length-matched training. Even with only 32-token test samples, the classifier maintains accuracy slightly below 70%—substantially above chance. This result has significant practical implications, as it enables a single model trained on longer sequences to be deployed for fingerprinting arbitrary-length responses in the wild, including very short text snippets where collecting training data at every possible length would be impractical.

3.6. Generalization to LLM distillation models

A critical question for LLM fingerprinting is whether a model distilled from a teacher LLM retains the fingerprint of its teacher or inherits characteristics of its base architecture. To investigate this, we conduct a controlled distillation experiment where we fine-tune `meta-llama/Llama-3.2-1B-Instruct` using Low-Rank Adaptation (LoRA) (Hu et al., 2021) on responses generated by different teacher models: GPT-3.5 turbo, GPT-4.1, Claude 3.5 Haiku, and Claude 4.5 Sonnet. We create four separate distilled variants, each trained exclusively on outputs from one teacher model.

The LoRA configuration applies low-rank updates to all attention and feed-forward projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with rank $r = 8$, scaling parameter $\alpha = 16$,

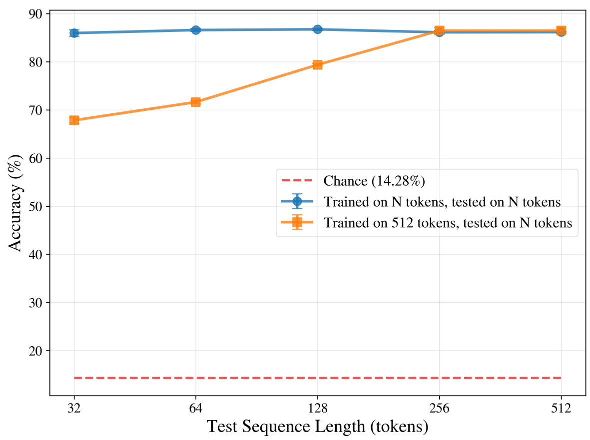

Figure 4: **Classification accuracy as a function of test sequence length under two training regimes.** Blue: classifier trained and tested on matching sequence lengths. Orange: classifier trained on 512-token sequences and tested on varying lengths. The red dashed line indicates random chance (14%). Error bars show standard deviation. A single classifier trained on long sequences generalizes effectively to shorter test sequences.

and dropout rate of 0.05. We employ RSLoRA (Kalajdzievski, 2023) for improved training stability. After distillation, we generate responses from each distilled model using the same prompt dataset, encode them into 384-dimensional representations, and evaluate them with our pre-trained classifier.

To create a more challenging and realistic test scenario, we also train a modified version of the classifier that includes responses from the base model `Meta Llama 3.2-1B` as an additional class, alongside the original six LLM classes. This setup poses a fundamental question: **Will the distilled models be classified as their base architecture (Llama) or as their teacher models (GPT/Claude)?**

This modified classifier model shows robust behavior similar to the models reported in Figure 5. Figure 6 presents the classification results using this classifier model for 700 test samples from each distilled model. Strikingly, the classifier predominantly assigns the distilled models to their respective *teacher* classes rather than the base Llama architecture. These results demonstrate that behavioral fingerprints induced through distillation are dominated by the training data and teacher model characteristics rather than the underlying base architecture.

4. Discussion

Our results demonstrate that LLM-generated text contains robust, model-specific fingerprints that persist across different conditions and can be detected with high accuracy using semantic embeddings and supervised learning. We discuss several key findings and their implications.

4.1. Robustness of LLM fingerprints

The high classification accuracy of approximately 89% across seven distinct LLM families/versions (Figure 1) indicates that each model exhibits characteristic generation patterns that are consistently encoded in the semantic space and classified using a fairly light-weight classifier. Importantly, our generalization experiments reveal that these fingerprints are stable across model versions: unseen variants such as GPT 3.5 Turbo (0125) and GPT 4.1 (Latest) are correctly mapped to their respective model families despite never being encountered during training. This version-invariance suggests that the learned representations capture fundamental properties of each model's architecture, training methodology, or alignment procedures, rather than superficial artifacts specific to particular releases.

The stability of these fingerprints likely stems from persistent differences in how models balance various objectives during training—such as factual accuracy, coherence, safety alignment, and stylistic preferences—which collectively shape their generation distributions. Even as models are updated or fine-tuned, core architectural choices and foundational training paradigms appear to leave detectable traces in the semantic structure of outputs.

4.2. Performance improvement with test-time scaling

In practical deployment scenarios, LLM fingerprinting is often applied to contexts where multiple samples from the same source model are available—such as conversation threads, document collections, or repeated interactions with an API. Our accuracy@k analysis (Fig. 3, left panel) demonstrates that classification performance improves substantially when aggregating predictions across multiple samples from the same model. Even in challenging cases such as distinguishing between Claude Sonnet 4.5 and Claude Opus 4.1—where single-sample accuracy is relatively low due to high similarity between these models—multi-sample aggregation enables reliable attribution. The steady improvement with increasing k suggests that classification errors on individual samples are not systematic but rather reflect inherent ambiguity in borderline cases, which can be resolved through statistical aggregation.

Importantly, this test-time scaling property generalizes to out-of-distribution scenarios. When evaluating responses to common everyday prompts generated by GPT-5.1 (a prompt distribution not seen during training) and with varied temperature settings, we observe similarly robust convergence toward 100% accuracy with increasing k (Fig. 3, center panel). This demonstrates that the fingerprinting approach

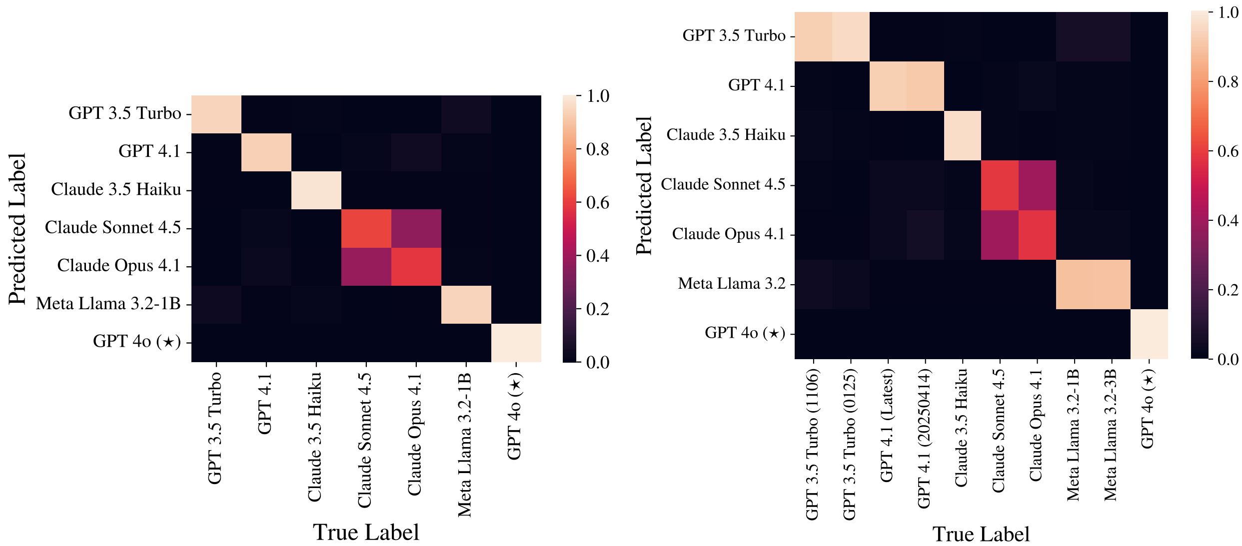

Figure 5: **Modified classifier (with Llama) confusion matrix. Left:** Results when tested only on model classes encountered during training. **Right:** Results when also tested on held-out model classes (GPT-3.5 Turbo (0125), GPT-4.1 (Latest), Meta Llama 3.2-3B)

is not overfitted to the specific prompt distribution or generation parameters used during training, but rather captures stable behavioral signatures that persist across diverse sampling conditions. The practical implication is that deployed fingerprinting systems can achieve near-perfect accuracy in realistic multi-sample scenarios, even when individual prompts or generation settings differ from training conditions.

4.3. Minimal text requirements for fingerprinting

Our sequence length ablation studies (Figure 4) reveal a surprising finding: LLM fingerprints manifest remarkably early in the generation process. A classifier trained and tested on matched sequence lengths maintains approximately 86–87% accuracy even for 32-token responses, demonstrating that discriminative features emerge within the first few tokens. Furthermore, a single classifier trained on 512-token sequences can effectively fingerprint arbitrary-length responses at test time, achieving 79–86% accuracy on sequences as short as 128 tokens and maintaining above-chance performance (68%) even on 32-token snippets.

The surprisingly strong performance even on very short sequences raises an important question: **why do model-specific fingerprints manifest so early in the generation process?** We hypothesize that this phenomenon is partially attributable to the architecture of the sentence encoding model used for embedding. The MiniLM encoder incorporates pooling layers that aggregate information across all tokens in the input sequence before projecting to the 384-dimensional representation. This pooling operation computes holistic, sequence-level statistics, such as mean or attention-weighted combinations of token representations, rather than treating tokens independently. Consequently, even short sequences may encode sufficient distributional information to reveal model-specific patterns. Different LLMs exhibit distinct preferences in lexical choice, syntactic structure, and semantic framing from the very first tokens.

This interpretation is further supported by the test-time generalization results, where a classifier trained on 512-token embeddings can still extract meaningful fingerprints from much shorter test sequences. If fingerprints depended on the accumulation of specific long-range patterns or required an extensive context, we would expect more severe performance degradation. Instead, the graceful decline suggests that the sentence encoder's pooling-based aggregation provides a length-invariant summary of generation characteristics that remains informative even in truncated sequences.

This has significant practical implications: LLM fingerprinting does not require lengthy documents or complete responses. Short text fragments, such as individual sentences or even partial sentences, contain sufficient information for model attribution. This enables deployment scenarios where only limited text samples are available, such as detecting AI-generated content in social media posts, chat messages, or code snippets. The graceful degradation of performance with decreasing sequence length also suggests that the method can be adapted to different use cases by trading off accuracy against minimum text requirements.

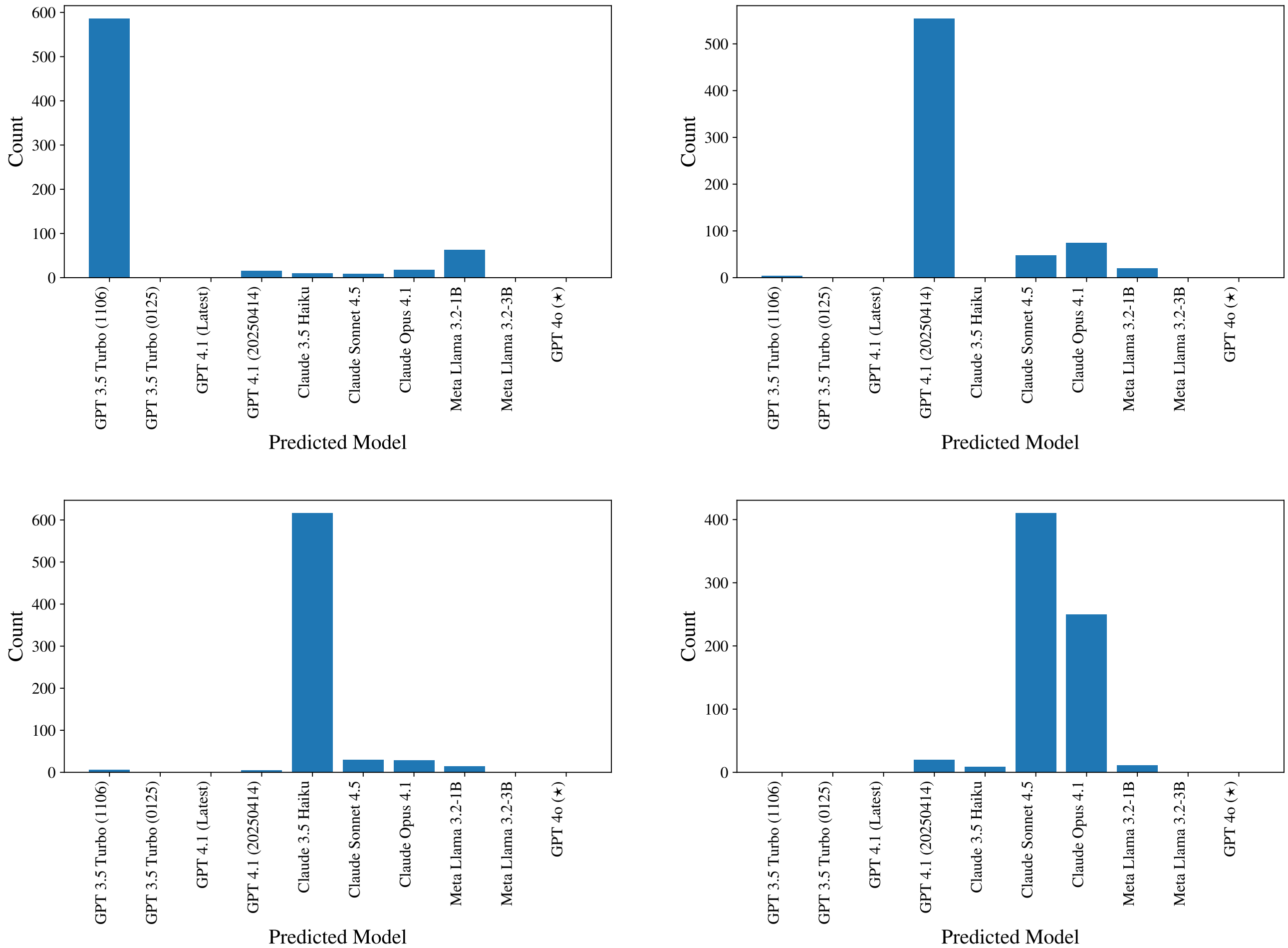

Figure 6: **Distilled model fingerprints.** Classification results for Llama-3.2-1B models distilled on different teacher LLMs. **(Top-left)** Distilled on GPT-3.5 Turbo (1106). **(Top-right)** Distilled on GPT-4.1 (20250414). **(Bottom-left)** Distilled on Claude 3.5 Haiku. **(Bottom-right)** Claude Sonnet 4.5. Each plot shows the distribution of predicted classes for 700 test samples.

4.4. Distillation and teacher behavior fingerprints

Perhaps the most striking finding emerges from our distillation experiments: when a Llama-based model is fine-tuned on responses from a teacher model (GPT or Claude variants), the resulting distilled model is classified predominantly as the teacher rather than the base architecture.

This demonstrates that behavioral fingerprints are strongly influenced—and potentially dominated—by training data and fine-tuning procedures rather than solely by underlying architecture. The LoRA adapters, despite modifying only a small fraction of model parameters, are sufficient to impart the characteristic generation patterns of the teacher model.

This finding has important implications for model provenance and intellectual property: distillation and fine-tuning can effectively transfer behavioral signatures, making it possible to trace adapted models back to their training data sources or teacher models. From a security perspective, this raises concerns about model laundering: a developer could potentially distill a proprietary model's (e.g., GPT or Claude) behavior onto an open-source base (e.g., Llama) and claim independent development. Our fingerprint classifier tool is currently limited in detecting such adversaries and will require further work to address these limitations.

4.5. Implications for AI Safety and Governance

The ability to reliably fingerprint LLM outputs has several implications for AI safety, governance, and accountability. As AI-generated content becomes increasingly prevalent, attribution methods can help establish content provenance and combat misinformation. While watermarking approaches have been proposed, behavioral fingerprinting offers a complementary technique that does not require modification of the generation process and can be applied retroactively to existing content. Organizations and platforms can use fingerprinting to detect unauthorized model use, enforce terms of service, or identify policy violations (for instance, academic institutions could detect student submissions generated by specific commercial models). When harmful or problematic content is generated by an LLM, fingerprinting can help identify the responsible model provider, enabling appropriate accountability mechanisms and informing targeted safety improvements. Finally, in scientific research involving LLM outputs, fingerprinting can help verify that reported results were indeed generated by the claimed model, improving reproducibility and detecting fabricated data.

4.6. Limitations and future work

Several limitations warrant discussion. First, our approach relies on a fixed sentence encoder (384-dimensional embeddings), which may not capture all relevant stylistic or structural differences between models. Future work could explore alternative embedding methods, including embeddings from the LLMs themselves or multi-modal representations.

Second, our classifier is trained in a closed-world setting with a fixed set of known models. In practice, new models are continuously released, and the classifier would need periodic retraining. We show some promise in using our classifier to detect model families (such as GPTs and Claudes) and versions (e.g., GPT 3.5 Turbo vs 4.1). However, more work is needed to perform more fine-grained fingerprinting.

Third, adversarial robustness remains an open question. A motivated adversary with knowledge of the fingerprinting system could potentially develop evasion strategies, such as post-processing outputs to mimic other models or deliberately introducing noise to obscure fingerprints. Evaluating robustness against adaptive attacks is an important direction for future research.

Fourth, while our distillation experiments demonstrate behavioral inheritance, we have only tested a limited set of teacher-student pairs. More comprehensive studies across diverse architectures, model sizes, and distillation strategies would provide a fuller picture of how fingerprints transfer through model adaptation.

Finally, our analysis focuses on English text and a specific set of prompts. Investigating multilingual fingerprinting, domain-specific performance (e.g., code, creative writing, technical documents), and the influence of prompt characteristics would broaden the applicability of these methods.

5. Conclusion

We have presented a supervised learning approach for LLM fingerprinting based on semantic embeddings of generated text. Our method achieves high accuracy (89%) in attributing responses to their source models and demonstrates remarkable robustness across several challenging conditions: unseen model versions, varying length of text sequences, and models distilled from different teachers.

6. Acknowledgments

We thank Roy Rinberg for useful discussion about the project and sharing the initial codes that we built on top of.

References

Busch, F., Hoffmann, L., Rueger, C., van Dijk, E. H., Kader, R., Ortiz-Prado, E., Makowski, M. R., Saba, L., Hadamitzky, M., Kather, J. N., Truhn, D., Cuocolo, R., Adams, L. C., and Bressem, K. K. Current applications and challenges in large language models for patient care: a systematic review. *Communications Medicine*, 5:26, January 2025. ISSN 2730-664X. doi: 10.1038/s43856-024-00717-2. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC11751060/.

Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks : the official journal of the International Neural Network Society*, 107:3–11, 2017. URL https://api.semanticscholar.org/CorpusID:6940861.

Fu, Z., Chen, J., Zhang, L., Yang, T., Niu, J., Sun, H., Li, R., Liu, P., Wang, J., He, F., and Zhang, Y. FDLLM: A Dedicated Detector for Black-Box LLMs Fingerprinting, June 2025. URL http://arxiv.org/abs/2501.16029. arXiv:2501.16029 [cs].

Gubri, M., Ulmer, D., Lee, H., Yun, S., and Oh, S. J. TRAP: Targeted Random Adversarial Prompt Honeypot for Black-Box Identification, June 2024. URL http://arxiv.org/abs/2402.12991. arXiv:2402.12991 [cs].

Hu, J. E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., and Chen, W. Lora: Low-rank adaptation of large language models. *ArXiv*, abs/2106.09685, 2021. URL https://api.semanticscholar.org/CorpusID:235458009.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.*, 43(2):42:1–42:55, January 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL https://doi.org/10.1145/3703155.

Jin, H., Zhang, C., Shi, S., Lou, W., and Hou, Y. T. ProFLingo: A Fingerprinting-based Intellectual Property Protection Scheme for Large Language Models, September 2024. URL http://arxiv.org/abs/2405.02466. arXiv:2405.02466 [cs].

Kalajdzievski, D. A rank stabilization scaling factor for fine-tuning with lora. *ArXiv*, abs/2312.03732, 2023. URL https://api.semanticscholar.org/CorpusID:266053836.

Kearney, M., Binns, R., and Gal, Y. Language Models Change Facts Based on the Way You Talk, July 2025. URL http://arxiv.org/abs/2507.14238. arXiv:2507.14238 [cs] version: 1.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. URL https://api.semanticscholar.org/CorpusID:6628106.

Meta. meta-llama/Prompt-Guard-86M · Hugging Face, December 2024. URL https://huggingface.co/meta-llama/Prompt-Guard-86M.

METR. Measuring AI Ability to Complete Long Tasks. *METR Blog*, March 2025. URL https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-

Nasery, A., Hayase, J., Brooks, C., Sheng, P., Tyagi, H., Viswanath, P., and Oh, S. Scalable Fingerprinting of Large Language Models. March 2025. URL https://openreview.net/forum?id=0wMUdbvYLL.

Nikolic, I., Baluta, T., and Saxena, P. Model Provenance Testing for Large Language Models, October 2025. URL http://arxiv.org/abs/2502.00706. arXiv:2502.00706 [cs].

Pasquini, D., Kornaropoulos, E. M., and Ateniese, G. LLMmap: Fingerprinting For Large Language Models, February 2025. URL http://arxiv.org/abs/2407.15847. arXiv:2407.15847 [cs].

Russinovich, M. and Salem, A. Hey, That's My Model! Introducing Chain & Hash, An LLM Fingerprinting Technique, June 2025. URL http://arxiv.org/abs/2407.10887. arXiv:2407.10887 [cs].

Tsai, Y.-Y., Guo, C., Yang, J., and Maaten, L. v. d. RoFL: Robust Fingerprinting of Language Models, May 2025. URL http://arxiv.org/abs/2505.12682. arXiv:2505.12682 [cs].

Turner, E., Soligo, A., Taylor, M., Rajamanoharan, S., and Nanda, N. Model Organisms for Emergent Misalignment, June 2025. URL http://arxiv.org/abs/2506.11613. arXiv:2506.11613 [cs].

van der Maaten, L. and Hinton, G. E. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL https://api.semanticscholar.org/CorpusID:5855042.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., and Zhou, M. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers, April 2020. URL http://arxiv.org/abs/2002.10957. arXiv:2002.10957 [cs].

Wen, Y., Zhang, K., Li, Z., and Qiao, Y. A discriminative feature learning approach for deep face recognition. In *European Conference on Computer Vision*, 2016. URL https://api.semanticscholar.org/CorpusID:4711865.

Xu, J., Wang, F., Ma, M., Koh, P. W., Xiao, C., and Chen, M. Instructional Fingerprinting of Large Language Models. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3277–3306, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.180. URL https://aclanthology.org/2024.naacl-long.180/.

Yang, Z. and Wu, H. A Fingerprint for Large Language Models, July 2024. URL http://arxiv.org/abs/2407.01235. arXiv:2407.01235 [cs].

Yu, S., Luo, M., Madasu, A., Lal, V., and Howard, P. Is Your Paper Being Reviewed by an LLM? Investigating AI Text Detectability in Peer Review, December 2024. URL http://arxiv.org/abs/2410.03019. arXiv:2410.03019 [cs] version: 2.

A. Examples from Common Prompts Dataset

#	Prompt Content
1	Explain the concept of inflation using simple examples involving grocery prices increasing over time.
2	Summarize the main points of the New York Times article published on March 14, 2024 about breakthroughs in Alzheimer's research.
3	Rewrite the following email so it sounds professional: 'Hey Mark, I still haven't gotten the report from you. What's going on?'
4	Correct the grammar in this paragraph from my scholarship essay: 'I always wanted to be someone who help people and make the world better place.'
5	Write a polite follow-up email asking for an update on the marketing coordinator position I interviewed for last Tuesday.
6	Suggest birthday gift ideas for my brother who is 27, loves hiking, photography, and jazz music.
7	Explain the difference between reinforcement learning and supervised learning using real-world examples such as game playing and image classification.
8	Help me brainstorm business ideas I can start with under $500 that use my skills in graphic design and social media marketing.
9	Write a short story about a retired astronaut who discovers a mysterious radio signal coming from their childhood home.
10	Explain photosynthesis step-by-step to help me prepare for my 10th-grade biology test.
11	Show me how to solve the calculus problem: 'Find the integral of $(4x^3–2x + 5)dx$.'
12	Translate this English sentence into Mandarin Chinese: 'I will arrive at the airport around noon.'
13	Improve this résumé bullet point: 'Worked on a team and completed tasks on time.'
14	Write a cover letter for a software engineering internship at Spotify using my experience with Python, Flask, and SQL.
15	Create a 20-minute full-body workout routine that requires no equipment and can be done in a small apartment.
16	Make a 7-day vegetarian meal plan with 1,800 calories per day and two snacks included daily.
17	Provide a simple recipe for homemade tomato soup using canned tomatoes, garlic, onions, and herbs.
18	Suggest conversation starters I can use when meeting my girlfriend's parents for the first time during dinner.
19	Optimize this Python function that sorts a large list of numbers: 'def slow_sort(nums): for i in range(len(nums)): ...'
20	Diagnose why this JavaScript code returns 'undefined' instead of the expected object: 'function getUser(){ let user = {name: "Lena", age: 24}; return; user; }'
21	Explain how the Dijkstra shortest path algorithm works using a road network with cities and travel times as the example graph.
22	Suggest five beginner-friendly programming project ideas I can build to learn TypeScript.
23	Interpret the PyTorch error message: 'RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and cpu.'
24	Write a Python function that checks whether the string 'racecar' is a palindrome by comparing characters from each end.
25	Give me 10 data analyst interview questions and provide sample answers suitable for someone with two years of experience.
26	Generate 20 unique brand name ideas for a handmade scented candle business inspired by nature.
27	Provide an outline for a research paper examining the economic impact of rising sea levels on coastal U.S. cities.
28	List the pros and cons of electric vehicles for both consumers and manufacturers.
29	Summarize the key findings of the 2022 Nature paper on CRISPR gene editing that discusses off-target mutations.
30	Create a 30-day step-by-step plan to learn HTML, CSS, and JavaScript from scratch.

B. Dataset Generation Details

B.1. Prompt Templates

1. Explain the concept of {}
2. Write a story about {}
3. Describe the history of {}
4. What are the benefits of {}?
5. How does {} work?
6. The future of {} is
7. A beginner's guide to {}
8. The importance of {} in modern society
9. Interesting facts about {}
10. The relationship between {} and technology
11. Common misconceptions about {}
12. The science behind {}
13. Why {} matters
14. The evolution of {}
15. Innovations in {}
16. The impact of {} on daily life
17. Understanding {} from first principles
18. The art and science of {}
19. A deep dive into {}
20. The basics of {}
21. Advanced techniques in {}
22. The philosophy of {}
23. Practical applications of {}
24. The cultural significance of {}
25. Recent developments in {}
26. The challenges of {}
27. Exploring {} through different perspectives
28. The ethics of {}
29. How to get started with {}
30. The beauty of {}

B.2. Topic List

- artificial intelligence
- cooking
- travel
- history
- science
- technology
- music
- art
- sports
- medicine
- economics
- philosophy
- literature
- mathematics
- astronomy
- biology
- chemistry
- physics
- psychology
- sociology
- education
- environment
- climate
- politics
- business
- architecture
- fashion
- photography
- cinema
- theater
- dance
- poetry
- mythology
- archaeology
- geology
- oceanography
- meteorology
- engineering
- robotics
- space exploration
- quantum mechanics
- genetics
- neuroscience
- machine learning
- cryptography
- nutrition
- fitness
- meditation
- gardening
- wildlife
- linguistics
- anthropology
- blockchain
- cybersecurity
- game design
- virtual reality
- augmented reality
- nanotechnology
- biotechnology
- renewable energy
- sustainable agriculture
- urban planning
- ecology
- marine biology
- astrophysics
- AI safety
- epidemiology
- pharmacology
- forensic science
- cognitive science
- ethics
- AI alignment
- social work
- public health
- international relations
- law
- criminology
- human geography
- cartography
- animation
- sound engineering
- graphic design
- industrial design
- web development
- software engineering
- data science
- statistics
- microbiology
- zoology
- botany
- paleontology
- hydrology
- energy policy
- supply chain management
- entrepreneurship
- marketing
- communication studies
- public speaking
- human–computer interaction
- computational linguistics
- bioinformatics
- systems biology
- tissue engineering
- materials science
- quantitative finance
- behavioral economics
- neuroeconomics
- artificial life
- exoplanet research
- planetary science
- stellar evolution
- geophysics
- volcanology
- seismology
- glaciology
- conservation biology
- environmental engineering
- forestry
- agroecology
- horticulture
- urban sociology
- cultural studies
- media studies
- gender studies
- queer theory
- critical race theory
- semiotics
- rhetoric
- folklore studies
- music theory
- ethnomusicology
- choreography
- set design
- creative writing
- screenwriting
- storytelling
- string theory
- neurology
- ornithology
- herpetology
- biomechanics
- computational physics
- theoretical biology
- information theory
- complex systems
- emergent behavior
- distributed systems